



저작자표시-비영리-변경금지 2.0 대한민국

이용자는 아래의 조건을 따르는 경우에 한하여 자유롭게

- 이 저작물을 복제, 배포, 전송, 전시, 공연 및 방송할 수 있습니다.

다음과 같은 조건을 따라야 합니다:



저작자표시. 귀하는 원저작자를 표시하여야 합니다.



비영리. 귀하는 이 저작물을 영리 목적으로 이용할 수 없습니다.



변경금지. 귀하는 이 저작물을 개작, 변형 또는 가공할 수 없습니다.

- 귀하는, 이 저작물의 재이용이나 배포의 경우, 이 저작물에 적용된 이용허락조건을 명확하게 나타내어야 합니다.
- 저작권자로부터 별도의 허가를 받으면 이러한 조건들은 적용되지 않습니다.

저작권법에 따른 이용자의 권리는 위의 내용에 의하여 영향을 받지 않습니다.

이것은 [이용허락규약\(Legal Code\)](#)을 이해하기 쉽게 요약한 것입니다.

[Disclaimer](#)

Master's Thesis

Deep Reinforcement Learning with State Restoration
Model for Bridging the Sim-to-Real Gap in
Two-Wheeled Bipedal Robot

Jae hyung Cho (조 재 형)

Department of Convergence IT Engineering

Pohang University of Science and Technology

2026



상태 복원 모델 기반 심층 강화학습 제어기를 이용한
이론-이족 로봇의 시뮬레이션-현실 격차 극복

Deep Reinforcement Learning with State Restoration
Model for Bridging the Sim-to-Real Gap in
Two-Wheeled Bipedal Robot

Deep Reinforcement Learning with State Restoration Model for Bridging the Sim-to-Real Gap in Two-Wheeled Bipedal Robot

by

Jae hyung Cho

Department of Convergence IT Engineering
Pohang University of Science and Technology

A thesis submitted to the faculty of the Pohang University of Science and
Technology in partial fulfillment of the requirements for the degree of Master
of Science in the IT Engineering

Pohang, Korea

12. 19. 2025

Approved by

Soohee Han (Signature)

Academic advisor



Deep Reinforcement Learning with State Restoration Model for Bridging the Sim-to-Real Gap in Two-Wheeled Bipedal Robot

Jae hyung Cho

The undersigned have examined this thesis and hereby certify that it is worthy
of acceptance for a master's degree from POSTECH

12. 19. 2025

Committee Chair Soohye Han (Seal)

Member Sangwoo Kim (Seal)

Member Poogyeon Park (Seal)



MCITE 조 재 형 Jae hyung Cho
20242680 Deep Reinforcement Learning with State Restoration Model for Bridging the
Sim-to-Real Gap in Two-Wheeled Bipedal Robot.
상태 복원 모델 기반 심층 강화학습 제어기를 이용한 이륜-이족 로봇의 시
뮬레이션-현실 격차 극복.
Department of Convergence IT Engineering , 2026, 29p
Advisor : Soohye Han.
Text in English.

ABSTRACT

Deploying locomotion policies on real robots remains challenging because onboard sensing provides only partial observations and because policies trained in simulation often degrade on hardware due to the sim-to-real gap. This thesis investigates sim-to-real deep reinforcement learning for a two-wheeled bipedal robot performing hybrid wheeled–legged locomotion, where coupled wheel–leg dynamics and intermittent contacts induce partial observability, making instantaneous measurements insufficient to form a Markovian state for stable control.

We propose a State Restoration Model (SRM) that reconstructs an enriched, state-like latent representation from observation histories. The SRM is trained to approximate privileged full-state information available in simulation and is integrated into an asymmetric actor–critic training pipeline: the actor is conditioned on the restored representation, while the critic leverages privileged information during training.

We evaluate the proposed approach on FlaminGO, an 8-DoF two-wheeled bipedal robot, following a simulation → simulation → real pipeline: training in Isaac Sim with Isaac Lab, cross-simulator validation in MuJoCo, and deployment on hardware. Across cross-simulator and real-world experiments, SRM improves commanded-velocity tracking and reduces steady-state bias compared with a baseline trained without SRM, including more stable station-keeping behavior under near-zero commands. These results

suggest that restoring a state-like representation from temporal context can mitigate ambiguity induced by partial observability and improve sim-to-real transfer robustness for hybrid locomotion.

Contents

I. Introduction	1
II. Preliminaries	3
2.1 Reinforcement Learning	3
2.2 Two-Wheeled Bipedal Robot Design	3
2.3 Sim-to-Real Transfer	5
III. Method	6
3.1 State Restoration Model	6
3.1.1 Architecture and Training Process	6
3.1.2 Learning Architecture Configuration	7
3.1.3 Learning Hybrid Locomotion Policy	10
3.1.4 Hybrid Locomotion Policy Configuration	11
IV. Experiment	14
4.1 Experimental Setup	14
4.1.1 Simulation → Simulation → Real Pipeline	14
4.1.2 Real-World Measurement Setup	15
4.1.3 Evaluation Protocol	15
4.2 Experimental Results	16
4.2.1 Training Performance Comparison	16
4.2.2 Simulation-to-Simulation Results (Isaac Sim → MuJoCo)	16
4.2.3 Real-World Velocity Tracking Performance	19
V. Conclusion	22
VI. Appendix	23
6.1 Training Hyperparameters	23
6.2 Real-World Full Trajectories	24
6.3 Pseudocode for SRM-PPO Training	25
Summary (in Korean)	27



List of Tables

2.1 Component Specifications of the Robot	4
3.1 State Composition and Observability	9
3.2 Domain randomization parameters and configuration for HLP	12
3.3 Reward Functions and Weights for the Hybrid Locomotion Policy	13
4.1 Velocity tracking performance for step input commands (sim-to-sim).	17
4.2 Real-world forward-velocity tracking errors.	19
4.3 Real-world station-keeping errors under zero command	21
6.1 Key training hyperparameters (network and optimization).	23

List of Figures

1.1	The FlaminGO robot: A novel multi-modal locomotion robot with hybrid wheeled and legged mechanisms. Each side of the robot consists of 4 degrees of freedom (DoF)—hip, shoulder, leg, and wheel joints—resulting in a total of 8 DoF.	2
3.1	Overview of the State Restoration Model (SRM) architecture. The observable state s_t^o represents sensor-accessible data in real-world scenarios, including joint positions, velocities, angular velocity, Euler angles, previous actions, and commands. In simulation, the privileged full state s_t^f encompasses additional information such as contact forces, joint torques, base height, and environmental features. During training, the GRU-based encoder maps s_t^o to a reconstructed state $\hat{s}_t^f$, approximating s_t^f . The actor network operates on $\hat{s}_t^f$, leveraging enriched state information to bridge the sim-to-real gap and enable robust decision-making. The critic network, using s_t^f , evaluates the policy while the SRM minimizes both reconstruction loss and reward consistency loss to align $\hat{s}_t^f$ with s_t^f .	9
3.2	Overview of the Hybrid Locomotion Policy learning framework. The SRM reconstructs the full state ($\hat{s}_t^f$) from simulator observations and provides it as input to the Hybrid Locomotion Policy, which operates at 50 Hz. The policy network is a fully connected neural network comprising three hidden layers with 512, 256, and 128 ReLU-activated nodes. This network outputs joint-level position and velocity targets ($q_1, q_2, \dots, q_8, \dot{q}_8$), facilitating robust hybrid locomotion by dynamically switching between walking and rolling locomotion to adapt to varying terrains.	10
4.1	Evaluation pipeline: training in simulation (Isaac Sim with Isaac Lab), cross-simulator validation (MuJoCo), and final deployment on the real robot.	15
4.2	Training performance comparison of policies with and without SRM. The top plot shows the mean episodic reward, and the bottom plot shows the forward velocity tracking reward (v_x).	17
4.3	Forward velocity response to step input commands with and without SRM (sim-to-sim), showing reduced steady-state error with SRM.	18
4.4	Representative real-world forward-velocity tracking results. Full trajectories for additional trials are provided in Appendix 6.2.	20

4.5 Real-world station-keeping under zero command ($v_x^{\text{cmd}} = 0$).	21
6.1 Full real-world trajectories used for qualitative reference.	24

I. Introduction

Deploying locomotion policies on real robots is challenging because onboard observations often do not fully represent the underlying state required for stable control. This *partial observability* is commonly formalized as a partially observable Markov decision process (POMDP), where robust decision-making generally requires integrating information over time rather than relying on a single instantaneous observation [1]. In addition, policies trained in simulation frequently degrade on hardware due to the *sim-to-real gap*, which arises from discrepancies such as unmodeled dynamics, sensor noise, delays, actuator bandwidth limits, and parameter variations [2, 3]. Together, partial observability and sim-to-real mismatch pose fundamental obstacles to reliable real-world locomotion [4, 5].

These obstacles are particularly salient for hybrid locomotion systems that combine wheels and legs. While wheels provide efficient traversal on flat terrain and legs can negotiate discontinuities such as steps and rough ground, combining rolling dynamics with intermittent contacts yields coupled behaviors and transition scenarios that are sensitive to sensing and terrain variations [6]. As a result, many hybrid systems rely on mode-dependent heuristics (e.g., separate rolling and stepping controllers) that require extensive tuning and can degrade performance at transitions [6]. Model-based approaches such as Model Predictive Control (MPC) [7, 8] further depend on accurate models of contact, friction, actuator dynamics, and latency, and they can be sensitive to modeling errors and unanticipated environment variations.

Deep reinforcement learning (DRL) has emerged as an alternative for learning locomotion behaviors with reduced manual crafting, demonstrating simulation-to-hardware transfer in legged systems [4, 5]. In typical pipelines, a policy is optimized in simulation (often with PPO [9]) and then deployed on hardware. However, robust deployment still hinges on addressing both sim-to-real mismatch and partial observability [2, 3, 1].

To mitigate sim-to-real mismatch, prior work introduces variability during training via domain randomization [2] and dynamics randomization [3]. Partial observability, meanwhile, motivates leveraging temporal context, and recurrent or memory-based RL methods have been used to summarize observation histories in partially observable environments [10]. Building on these ideas, we develop a DRL-based sim-to-real framework for hybrid locomotion that combines domain/dynamics randomization [2, 3] with a **State Restoration Model (SRM)**. SRM reconstructs an enriched, state-like latent representation from observation histories by integrating current sensory inputs with temporal context, reducing ambiguity when observations alone are insufficient to infer a Markovian state [1, 10].

We evaluate the proposed framework on **FlaminGO**, a multi-modal locomotion robot inspired by



Figure 1.1: The FlaminGO robot: A novel multi-modal locomotion robot with hybrid wheeled and legged mechanisms. Each side of the robot consists of 4 degrees of freedom (DoF)—hip, shoulder, leg, and wheel joints—resulting in a total of 8 DoF.

the joint structure of flamingos (Fig. [1.1](#)). FlaminGO integrates wheeled mobility with leg articulation for negotiating stairs and uneven ground. Each side consists of 4 degrees of freedom resulting in a total of 8 DoF. The actuation system uses only 8 actuators, compared to the 12 typically found in quadrupedal robots [\[11, 12, 13\]](#).

The contributions of this work are summarized as follows:

- **Multi-modal robot design:** We design and implement FlaminGO, an 8-DoF two-wheeled bipedal robot that combines wheeled mobility and leg articulation for hybrid locomotion across diverse terrains.
- **DRL-based sim-to-real framework:** We establish a simulation training and transfer pipeline for hybrid locomotion and incorporate domain/dynamics randomization to reduce sim-to-real sensitivity [\[2, 3\]](#).
- **State restoration under partial observability:** We propose SRM to reconstruct a state-like latent representation from observation histories, improving robustness when observations alone do not fully represent the underlying state [\[1, 10\]](#).

II. Preliminaries

2.1 Reinforcement Learning

To enable hybrid locomotion for the FlaminGO robot, we model the control problem as a Markov Decision Process (MDP) defined by the tuple (S, A, P, R, γ) . Here, S represents the robot's kinematic and dynamic states, A denotes the action space comprising motor commands, $P(s'|s, a)$ describes the transition dynamics, $R(s, a)$ is the reward function that evaluates the desirability of a given state-action pair, and γ is the discount factor balancing immediate and future rewards. At each time step t , the robot observes its state s_t , selects an action a_t following policy $\pi(a|s)$, transitions to a new state s_{t+1} , and receives a reward r_t . The objective is to optimize the policy π^* to maximize the expected cumulative reward.

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right], \quad (2.1)$$

where γ is the discount factor that balances immediate and future rewards.

To address the control problem formulated as an MDP, we employ Proximal Policy Optimization (PPO) [9], a policy-gradient algorithm known for its balance of stability and efficiency in learning. PPO iteratively refines the policy by constraining updates within a safe range, ensuring steady progress during training. Its clipping mechanism mitigates the risk of unstable updates, making it particularly suitable for complex robotic tasks. The PPO objective function is given by:

$$\mathcal{L}^{PPO}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (2.2)$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ represents the probability ratio, $\hat{A}_t$ is the advantage estimate, and ϵ is the clipping parameter.

2.2 Two-Wheeled Bipedal Robot Design

FlaminGO is a two-wheeled bipedal robot designed for hybrid locomotion by combining wheeled mobility with leg articulation. Each side of the robot has four actuated joints (hip, shoulder, leg, and wheel), resulting in a total of 8 DoF. The hip and shoulder joints are actuated by Robstride RS03 [14] units with a gear reduction ratio of 9:1, providing a rated torque of 20.0 Nm and a peak torque of 60.0 Nm. The leg joint uses a Robstride RS03 with an external gear module (gear reduction ratio 13.5:1), with a rated torque of 30.0 Nm and a peak torque of 90.0 Nm. For wheeled propulsion, the wheel joint is driven

by a Robstride RS06[14] actuator with a 9:1 gear reduction ratio, delivering a rated torque of 12 Nm and a peak torque of 36 Nm. The onboard computation is performed on an NVIDIA Jetson Orin Nano 8GB single-board computer (SBC), and inertial sensing is provided by an AHRS EBIMU-9DOFV5-R3 IMU. The robot is powered by a Samsung SDI 21700 50S battery pack configured as 13S1P.

The mechanical design targets hybrid locomotion scenarios in which the robot can roll efficiently on flat terrain and step over discontinuities such as stairs or uneven ground by coordinating leg articulation and wheel actuation. Table 2.1 summarizes the key components and specifications. The base weight of the platform is 10.5 kg, and the total weight including all components is 18.5 kg.

Table 2.1: Component Specifications of the Robot

Component Name	Specification
Joint Actuators (Hip, Shoulder)	Robstride RS03 Rated Torque: 20.0 Nm Peak Torque: 60.0 Nm Gear Reduction Ratio: 9:1
Joint Actuators (Leg)	Robstride RS03 (W/ External Gear Module) Rated Torque: 30.0 Nm Peak Torque: 90.0 Nm Gear Reduction Ratio: 13.5:1
Joint Actuators (Wheel)	Robstride RS06 Rated Torque: 12 Nm Peak Torque: 36 Nm Gear Reduction Ratio: 9:1
Single Board Computer (SBC)	NVIDIA Jetson Orin Nano 8GB
Inertia Measurement Unit (IMU)	AHRS EBIMU-9DOFV5-R3
Battery	Samsung SDI 21700 50S 13s 1p
Base Weight	10.5 kg
Total Weight	18.5 kg

2.3 Sim-to-Real Transfer

Sim-to-real transfer aims to deploy policies trained in simulation to real-world robots by mitigating the *sim-to-real gap*, i.e., performance degradation caused by discrepancies between simulated and real dynamics (e.g., unmodeled effects, sensing noise, latency, and parameter variations). To reduce this gap, we employ the following techniques:

- **Domain Randomization:** We randomize physical parameters in simulation (e.g., mass properties, friction, actuator gains, and sensor noise) to train policies that generalize across plausible real-world conditions [2].
- **State Restoration Model (SRM):** To address partial observability, we reconstruct an enriched, state-like latent representation from observation histories using SRM. By integrating current sensory inputs with temporal context, SRM provides the policy with a compact surrogate state that is more informative than instantaneous observations alone, improving robustness under noisy and incomplete measurements.
- **Latent State Transformation:** We transform raw observations into a latent vector using a recurrent belief encoder that aggregates proprioceptive and exteroceptive signals over time. This latent representation supports state estimation under partial observability and enables the policy to incorporate noisy exteroceptive inputs in a structured manner [15].

By combining domain randomization with history-based latent state inference, the resulting policy is trained to tolerate dynamics mismatch and sensing limitations, supporting reliable deployment for hybrid locomotion. The experimental results in later sections evaluate how these components reduce the sim-to-real gap in real-world operation.

III. Method

3.1 State Restoration Model

In real-world scenarios, robots often operate in environments where only partial observations of the full state are available due to sensor limitations or unobservable dynamics. This partial observability can lead to suboptimal decision-making and hinder the performance of reinforcement learning (RL)-based control policies, especially when transferring them from simulation to reality. In simulation, privileged information—such as precise physical states or dynamics—can be readily accessed, but these are typically unavailable in real-world deployments. This disparity introduces a significant challenge known as the sim-to-real gap, where policies trained in simulation fail to generalize effectively to the real world.

To address this challenge, we employ an *Asymmetric Actor-Critic* framework[16], where the critic network benefits from privileged information during training to evaluate actions, while the actor network operates solely on observable states. Despite this improvement, the absence of privileged information during deployment can limit the actor’s performance.

As illustrated in Figure 3.1, the State Restoration Model (SRM) bridges this gap by reconstructing an enriched state, $\hat{s}_t^f$, from the observable state, s_t^o . This enriched state approximates the privileged full state, s_t^f , ensuring robust and consistent decision-making for the actor network in real-world environments. The SRM is trained using a combination of reconstruction and reward consistency losses, described in subsequent sections.

3.1.1 Architecture and Training Process

The GRU encoder processes the partially observable state to estimate the privileged observations and reconstruct the full state as follows:

$$s_t^p = \text{GRU}_{\text{encoder}}(s_t^o) \quad (3.1)$$

$$\hat{s}_t^f = (s_t^o) || (s_t^p), \quad (3.2)$$

where s_t^o represents the partially observable input at time t , and $\hat{s}_t^f$ is the reconstructed full state. The GRU encoder processes the sequential input s_t^o to capture temporal dependencies and generate the predicted hidden state s_t^p . The reconstructed full state $\hat{s}_t^f$ is then obtained by concatenating s_t^o and s_t^p , serving as an approximation of s_t^f .



The SRM is trained using a restoration-based loss function, which minimizes the discrepancy between the restored state $\hat{s}_t^f$ and the full state s_t^f . The restoration loss is defined as:

$$\mathcal{L}_{\text{reconstruction}}(\hat{s}_t^f, s_t^f) = \begin{cases} \frac{1}{2}(\hat{s}_t^f - s_t^f)^2 & \text{if } |\hat{s}_t^f - s_t^f| \leq 1.0, \\ 1.0(|\hat{s}_t^f - s_t^f| - 0.5) & \text{otherwise} \end{cases} \quad (3.3)$$

ensuring that the reconstructed state closely approximates the privileged information used by the critic network during training.

To further enhance task-specific performance, SRM incorporates a reward consistency loss. This loss minimizes the difference in rewards computed using the full state s_t^f and the reconstructed state $\hat{s}_t^f$, aligning the task-relevant features between the two states:

$$\mathcal{L}_{\text{reward}} = \|R(s_t^f, a_t) - R(\hat{s}_t^f, a_t)\|^2 \quad (3.4)$$

The total loss for training the SRM is expressed as:

$$\mathcal{L}_{\text{SRM}} = \mathcal{L}_{\text{reconstruction}} + \mathcal{L}_{\text{reward}} \quad (3.5)$$

3.1.2 Learning Architecture Configuration

The SRM utilizes distinct state representations for the critic and actor networks to address the asymmetry in available information during training and deployment. Table 3.1 summarizes the differences between the privileged observations (s_t^p) and the partially observable state (s_t^o) used as input to the SRM.

Observable(s_t^o) represents the information directly accessible from sensors in real-world scenarios, providing a partial view of the robot's state:

- *Joint positions*(q_i) : Represents the angular positions of the robot's joints, measured in radians (*rad*).
- *Joint velocities*($\dot{q}_i$): Denotes the angular velocity of each joint, providing information on rotational speed in radians per second (*rad/s*).
- *Base angular velocity*($\omega_{x,y,z}$): Measures the rotational speed of the robot's base along its axes, expressed in radians per second (*rad/s*).
- *Base Euler angles*($\phi_{x,y,z}$): Describes the robot's orientation using roll, pitch, and yaw angles in radians (*rad*).
- *Previous actions*(a_t) : Tracks the actions executed in the previous time step to offer temporal context for decision-making.

- *Commands*($cmd_{\dot{x}, \dot{y}, \omega_z}$): Represents the high-level velocity and directional commands provided to the robot.
- *History buffer*($\mathcal{H}_k$) : Stores the states from time step $t - k$ to the current time step t , providing temporal context for the robot’s decision-making process.

Privileged (s_t^p) is accessible only during simulation, this state provides richer information critical for policy optimization:

- *Base linear velocity*(v_x): Represents the forward velocity of the robot’s base along the x-axis, measured in meters per second (m/s).
- *Current reward* (R_t): Indicates the immediate reward received by the robot, reflecting the effectiveness of the current actions.
- *Lift mask* (L_i): A boolean sensor data that indicates whether the robot has detected a curb or step, used to adjust hybrid locomotion.
- *Base height*(p_z): Specifies the vertical position of the robot’s base relative to the ground, measured in meters (m).

By reconstructing $\hat{s}_t^f$ from s_t^o , the SRM aligns the state representations between the actor and critic, thereby reducing the disparity caused by the absence of privileged data. The reconstructed state $\hat{s}_t^f$ enables the actor network to make robust and consistent decisions in real-world environments. Additionally, while the SRM focuses on reconstructing the state, the role of the reward in shaping task-specific behaviors will be further detailed in Section 3.1.3. Finally, the complete SRM-PPO training procedure including rollout collection, SRM optimization (reconstruction and reward-consistency losses), and PPO policy/value updates is provided as pseudocode in Appendix 6.3.

Table 3.1: State Composition and Observability

State Component	Observable (s_t^o)	Privileged (s_t^p)
Joint position (q_i)	✓	
Joint velocity ($\dot{q}_i$)	✓	
Base angular velocity ($\omega_{x,y,z}$)	✓	
Base Euler angle ($\Theta_{x,y,z}$)	✓	
Previous actions (a_t)	✓	
Commands ($cmd_{\dot{x},\dot{y},\omega_z}$)	✓	
History buffer ($\mathcal{H}_k$)	✓	
Base linear velocity (v_x)		✓
Current reward (R_t)		✓
Lift mask (L_i)		✓
Base height (p_z)		✓

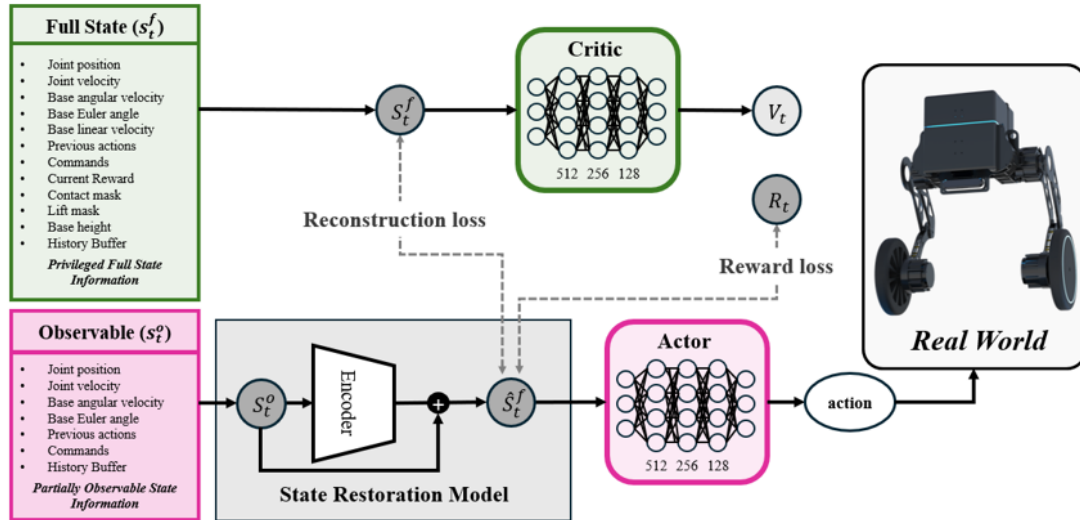


Figure 3.1: Overview of the State Restoration Model (SRM) architecture. The observable state s_t^o represents sensor-accessible data in real-world scenarios, including joint positions, velocities, angular velocity, Euler angles, previous actions, and commands. In simulation, the privileged full state s_t^f encompasses additional information such as contact forces, joint torques, base height, and environmental features. During training, the GRU-based encoder maps s_t^o to a reconstructed state $\hat{s}_t^f$, approximating s_t^f . The actor network operates on $\hat{s}_t^f$, leveraging enriched state information to bridge the sim-to-real gap and enable robust decision-making. The critic network, using s_t^f , evaluates the policy while the SRM minimizes both reconstruction loss and reward consistency loss to align $\hat{s}_t^f$ with s_t^f .

3.1.3 Learning Hybrid Locomotion Policy

HLP is designed to enable seamless hybrid locomotion by dynamically integrating walking and rolling modes. The policy is implemented as a fully connected neural network comprising three hidden layers with 512, 256, and 128 ReLU-activated nodes. It processes the reconstructed full state ($\hat{s}_t^f$) generated by the SRM to output joint-level position and velocity targets ($q_1, q_2, \dots, q_8$), facilitating robust and adaptive control.

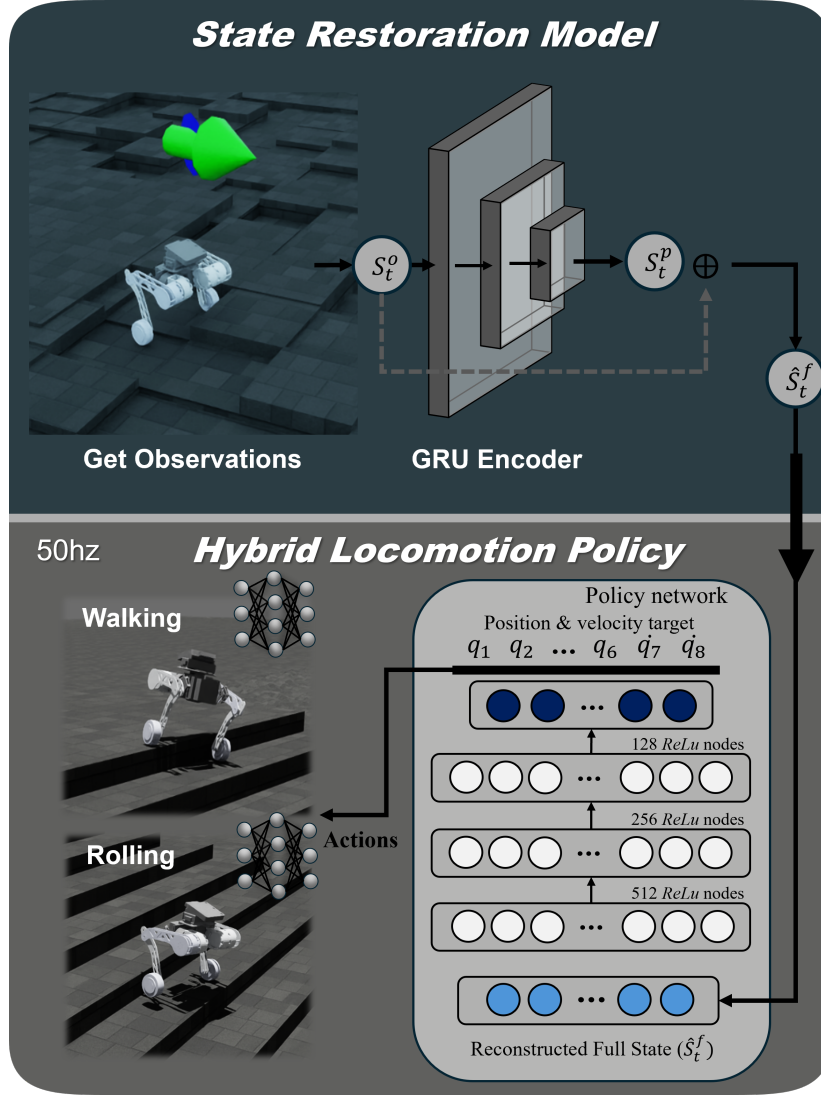


Figure 3.2: Overview of the Hybrid Locomotion Policy learning framework. The SRM reconstructs the full state ($\hat{s}_t^f$) from simulator observations and provides it as input to the Hybrid Locomotion Policy, which operates at 50 Hz. The policy network is a fully connected neural network comprising three hidden layers with 512, 256, and 128 ReLU-activated nodes. This network outputs joint-level position and velocity targets ($q_1, q_2, \dots, \dot{q}_8$), facilitating robust hybrid locomotion by dynamically switching between walking and rolling locomotion to adapt to varying terrains.

3.1.4 Hybrid Locomotion Policy Configuration

The Hybrid Locomotion Policy is configured to operate in both simulated and real-world environments by utilizing observations, applying domain randomization, and incorporating a structured reward mechanism. The key elements of the configuration are outlined as follows:

Observations and Domain Randomization

HLP utilizes observations derived from the robot's reconstructed full state, $\hat{s}_t^{f,H}$, as defined in the SRM architecture (see Section 3.1.2). A detailed breakdown of these observations, including their units and dimensions, is provided in Table 3.2

To enhance robustness and adaptability in real-world conditions, domain randomization is applied during training. By introducing variability to observation parameters and environmental dynamics, the HLP is exposed to a wide range of randomized MDPs. This approach reduces overfitting to simulated environments and equips the policy to handle unseen scenarios during deployment. The domain randomization parameters, covering both the robot's internal states and external environmental factors, are summarized in Table 3.2

To enhance robustness and adaptability in real-world conditions, domain randomization is applied during training. By introducing variability to observation parameters and environmental dynamics, the HLP is exposed to a wide range of randomized MDPs. This approach reduces overfitting to simulated environments and equips the policy to handle unseen scenarios during deployment. The domain randomization parameters, covering both the robot's internal states and external environmental factors, are summarized in Table 3.2

Reward Function

The reward function R_t is designed to optimize the HLP by addressing both task-specific objectives and regularization constraints. The total reward is expressed as:

$$R_t = r_{t,cmd_{x,y}}^{track} + r_{t,cmd_z}^{track} + r_{t,v_z}^{reg} + r_{t,\omega_{x,y}}^{reg} + r_{t,q_i}^{reg} + r_{t,\tau_i}^{reg} + r_{t,\dot{q}_i}^{reg} + r_{t,a_t}^{reg} + r_{t,p_z}^{reg}, \quad (3.6)$$

where $r_{t,cmd_{x,y}}^{track}$ tracks the desired linear velocity in the x and y directions, while r_{t,cmd_z}^{track} tracks the desired angular velocity around the z -axis for heading control. The regularization terms include r_{t,v_z}^{reg} , which penalizes vertical velocity to maintain stability, and $r_{t,\omega_{x,y}}^{reg}$, which penalizes angular velocities around the x and y axes to minimize unnecessary base rotations. Joint positions are regularized by r_{t,q_i}^{reg} to ensure they stay within allowable limits, while r_{t,τ_i}^{reg} penalizes excessive joint torques to promote energy efficiency. Further, $r_{t,\dot{q}_i}^{reg}$ penalizes joint accelerations to reduce mechanical strain, and r_{t,a_t}^{reg} regularizes

Table 3.2: Domain randomization parameters and configuration for HLP

Parameters	Dim	Unit	Range	Operator
q_i	6	rad	$[-0.04, 0.04]$	additive
$\dot{q}_i$	8	rad/s	$[-1.0, 1.0]$	additive
$\phi_{x,y,z}$	3	rad	$[-0.125, 0.125]$	additive
$\omega_{x,y,z}$	3	rad/s	$[-1.0, 1.0]$	additive
$v_{x,y,z}$	3	rad/s	$[-0.15, 0.15]$	additive
p_z	1	m	$[-0.05, 0.05]$	additive
τ_i	8	Nm	$[-0.25, 0.25]$	additive
$\ddot{q}_i$	8	rad/s ²	$[-1.75, 1.75]$	additive
H_t	64	m	$[-0.025, 0.025]$	additive
$cmd_{x,y,z}$	3	m/s	$[-2.0, 2.0]$	additive
$delay_{system}$	-	ms	$[0.0, 20.0]$	additive
$friction_{static}$	-	μ	$[0.3, 1.0]$	scaling
$friction_{dynamic}$	-	μ	$[0.6, 1.0]$	scaling
$mass_{body}$	-	kg	$[-1.25, 3.5]$	additive
$push$	-	s	$[10.0, 15.0]$	additive

the rate of change in actions to ensure smooth control inputs. Finally, r_{t,p_z}^{reg} encourages the robot's base height to remain within a stable range for better terrain adaptability.

The detailed mathematical definitions of these reward components are provided in Table 3.3.

Table 3.3: Reward Functions and Weights for the Hybrid Locomotion Policy

Reward	Equation (r_t)	Weight (w)	Params (ξ)
$r_{t,cmd_{x,y}}^{track}$	$w \cdot \exp(-\xi \cdot \ cmd_{x,y} - v_{x,y}\ ^2)$	2.0	4.0
$r_{t,cmd_{\omega_z}}^{track}$	$w \cdot \exp(-\xi \cdot \ cmd_{\omega_z} - \omega_z\ ^2)$	1.0	4.0
r_{t,v_z}^{reg}	$-w \cdot \ v_z\ ^2$	0.1	-
$r_{t,\omega_{x,y}}^{reg}$	$-w \cdot \ \omega_{x,y}\ ^2$	0.05	-
r_{t,q_i}^{reg}	$-w \cdot \ q_{2i} - \xi\ ^2, i \in \{0, 1, 2, 3\}$	0.2	q_{2i+1}
r_{t,q_i}^{reg}	$-w \cdot \ q_i - \xi\ ^2$	0.1	target
r_{t,τ_i}^{reg}	$-w \cdot \ \tau_i\ ^2$	0.05	-
$r_{t,\dot{q}_i}^{reg}$	$-w \cdot \ \dot{q}_i\ ^2$	0.001	-
r_{t,a_t}^{reg}	$-w \cdot \ a_t - a_{t-1}\ ^2$	$5.0 \cdot 10^{-4}$	-
r_{t,p_z}^{reg}	$-w \cdot \ p_z - \xi\ ^2$	20.0	0.40187

IV. Experiment

4.1 Experimental Setup

This section describes the evaluation pipeline and experimental settings used to validate the proposed SRM-based framework across (i) simulation training, (ii) cross-simulator transfer, and (iii) real-world deployment. Concretely, we follow a *simulation* $\rightarrow$ *simulation* $\rightarrow$ *real* procedure, where the policy is trained in a high-throughput simulator and then validated in an independent physics engine before deployment on the physical robot (Fig. 4.1). For simulation, we use Isaac Lab built on NVIDIA Isaac Sim [17] and evaluate transfer in MuJoCo [18].

4.1.1 Simulation $\rightarrow$ Simulation $\rightarrow$ Real Pipeline

Stage 1: Training in simulation. Training is performed in simulation to enable fast, safe, and scalable rollout generation. We train the locomotion policy using PPO [9] in Isaac Lab [17]; the key network architectures and optimization hyperparameters are summarized in Appendix 6.1. During training, we monitor learning progress using (i) the mean episodic reward and (ii) task-specific tracking reward terms (e.g., forward-velocity tracking). We additionally conduct an ablation by training two policies under identical settings: *with SRM* and *without SRM*. This allows us to compare learning dynamics (e.g., reward growth rate and final reward at a fixed number of steps) and quantify whether SRM improves optimization under partial observability.

Stage 2: Sim-to-sim validation. Before hardware deployment, the trained policy is evaluated in MuJoCo [18] to reduce simulator-specific overfitting and to verify that learned behaviors reproduce under a different physics stack (Fig. 4.1). This stage provides a controlled validation step for velocity tracking and stability tests prior to real-world experiments. In particular, we evaluate step-command tracking over a range of commanded forward velocities and compare SRM versus the baseline under identical command profiles.

Stage 3: Sim-to-real deployment. Finally, the policy is transferred to the physical robot and evaluated in real-world trials, focusing on commanded-velocity tracking and stability under practical sensing constraints.

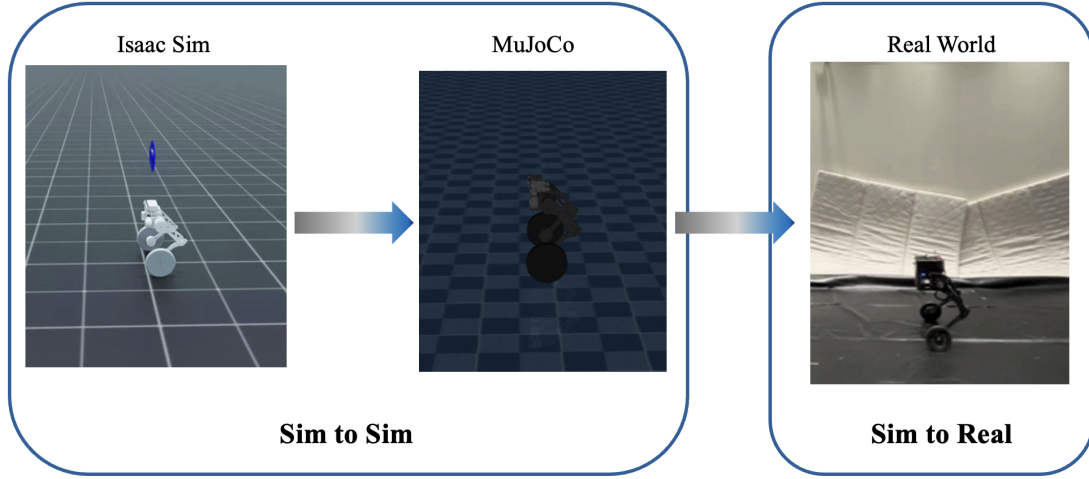


Figure 4.1: Evaluation pipeline: training in simulation (Isaac Sim with Isaac Lab), cross-simulator validation (MuJoCo), and final deployment on the real robot.

4.1.2 Real-World Measurement Setup

To quantify tracking performance on hardware, we estimate the robot’s linear velocity from external motion-capture measurements. Specifically, linear velocity is computed from OptiTrack marker positions using a first-order forward Euler difference:

$$\mathbf{v}(t_k) \approx \frac{\mathbf{p}(t_k) - \mathbf{p}(t_{k-1})}{\Delta t}, \quad \Delta t = t_k - t_{k-1}. \quad (4.1)$$

This procedure corresponds to the explicit (forward) Euler method for first-order numerical differentiation integration [19]. In our setup, the resulting velocity estimate has an approximate measurement error of 0.06 m/s.

4.1.3 Evaluation Protocol

Across both simulators and real-world experiments, we assess the controller primarily through commanded-velocity tracking tests using step-command scenarios. Each trial consists of (i) a commanded forward velocity profile and (ii) the measured forward velocity response. We compute tracking errors after time alignment between the command and measured signals.

Command profiles. We evaluate step commands spanning a representative range of forward velocities (e.g., 0.5–1.0 m/s in simulation) to test both transient response and steady-state regulation. The same class of step inputs is used for cross-simulator validation and real-world trials to keep evaluation conditions consistent across the pipeline.

Metrics. For each trial, we report time-domain tracking errors over a fixed evaluation horizon:

$$\text{RMSE} = \sqrt{\frac{1}{T} \sum_{k=1}^T (v_x(t_k) - v_x^{\text{cmd}}(t_k))^2}, \quad \text{MAE} = \frac{1}{T} \sum_{k=1}^T |v_x(t_k) - v_x^{\text{cmd}}(t_k)|. \quad (4.2)$$

To separate transient and steady-state behavior for step responses, we additionally compute *steady-state* metrics over a terminal window of fixed duration (e.g., the last 2 s of a trial), denoted as w_RMSE and w_MAE . This windowed evaluation reduces sensitivity to initial transients and emphasizes final tracking accuracy under constant command.

4.2 Experimental Results

We report experimental results following the *simulation* $\rightarrow$ *simulation* $\rightarrow$ *real* evaluation pipeline. Unless otherwise stated, we compare two policies trained under identical settings: a baseline trained *without SRM* and the proposed method trained *with SRM*. We evaluate (i) training performance, (ii) velocity tracking performance under simulation-to-simulation transfer (Isaac Sim $\rightarrow$ MuJoCo), and (iii) velocity tracking performance on the real robot.

4.2.1 Training Performance Comparison

To analyze the impact of SRM on policy learning, we compare the reward progression of policies trained *with* and *without* SRM under identical training settings. Figure 4.2 shows the evolution of the mean episodic reward and the tracking reward for forward linear velocity (v_x) over 10,000 training steps.

At 10,000 steps, the policy trained with SRM achieves a higher mean reward (48.03) than the policy trained without SRM (45.19). In addition, the v_x tracking reward is slightly higher with SRM (1.8347) than without SRM (1.8037). These results are consistent with SRM providing a more informative state-like representation from partial observations, which improves optimization and tracking-related reward terms.

4.2.2 Simulation-to-Simulation Results (Isaac Sim $\rightarrow$ MuJoCo)

We next report *simulation-to-simulation* results to assess cross-simulator robustness by executing the learned policy in an independent physics engine (MuJoCo). This evaluation reduces the risk of simulator-specific overfitting and focuses on commanded-velocity tracking under step inputs.

Velocity Tracking Performance

To quantitatively evaluate forward velocity tracking in the sim-to-sim setting, we conduct step-command tests at three commanded speeds (0.5, 0.75, and 1.0 m/s). We report the settling time and steady-state error in Table 4.1. Settling time is defined as the time required for the base forward velocity to remain within $\pm 5\%$ of the commanded velocity. Steady-state error is defined as the absolute difference between the steady-state base velocity and the commanded velocity.

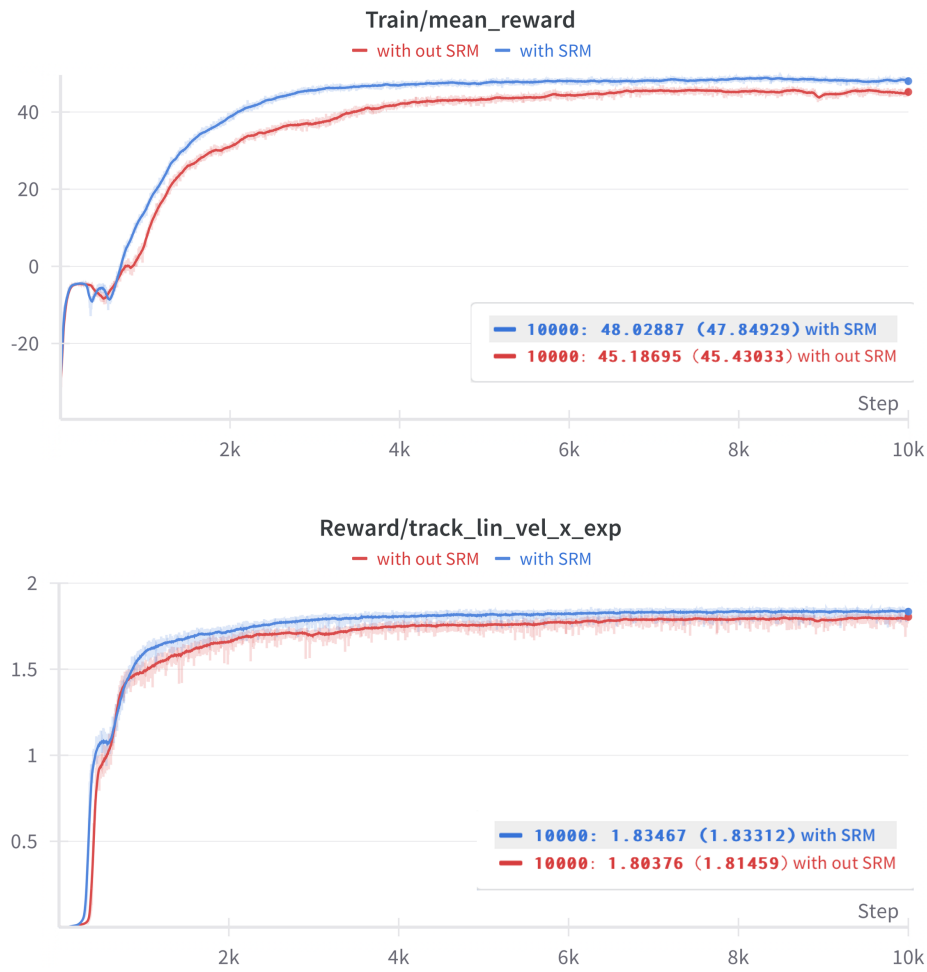


Figure 4.2: Training performance comparison of policies with and without SRM. The top plot shows the mean episodic reward, and the bottom plot shows the forward velocity tracking reward (v_x).

Table 4.1: Velocity tracking performance for step input commands (sim-to-sim).

Command (m/s)	Condition	Settling Time (s)	Steady-State Error (m/s)
0.50	with SRM	0.62	0.0002
	without SRM	0.50	0.1058
0.75	with SRM	0.68	0.0020
	without SRM	0.60	0.0771
1.00	with SRM	0.76	0.0039
	without SRM	0.74	0.0295

Across all tested speeds, SRM substantially reduces steady-state error. For example, at a commanded velocity of 0.5 m/s, the steady-state error decreases from 0.1058 m/s to 0.0002 m/s. At 1.0 m/s, the error decreases from 0.0295 m/s to 0.0039 m/s. Settling time increases slightly with SRM in these tests, indicating a more conservative transient response while achieving lower steady-state bias.

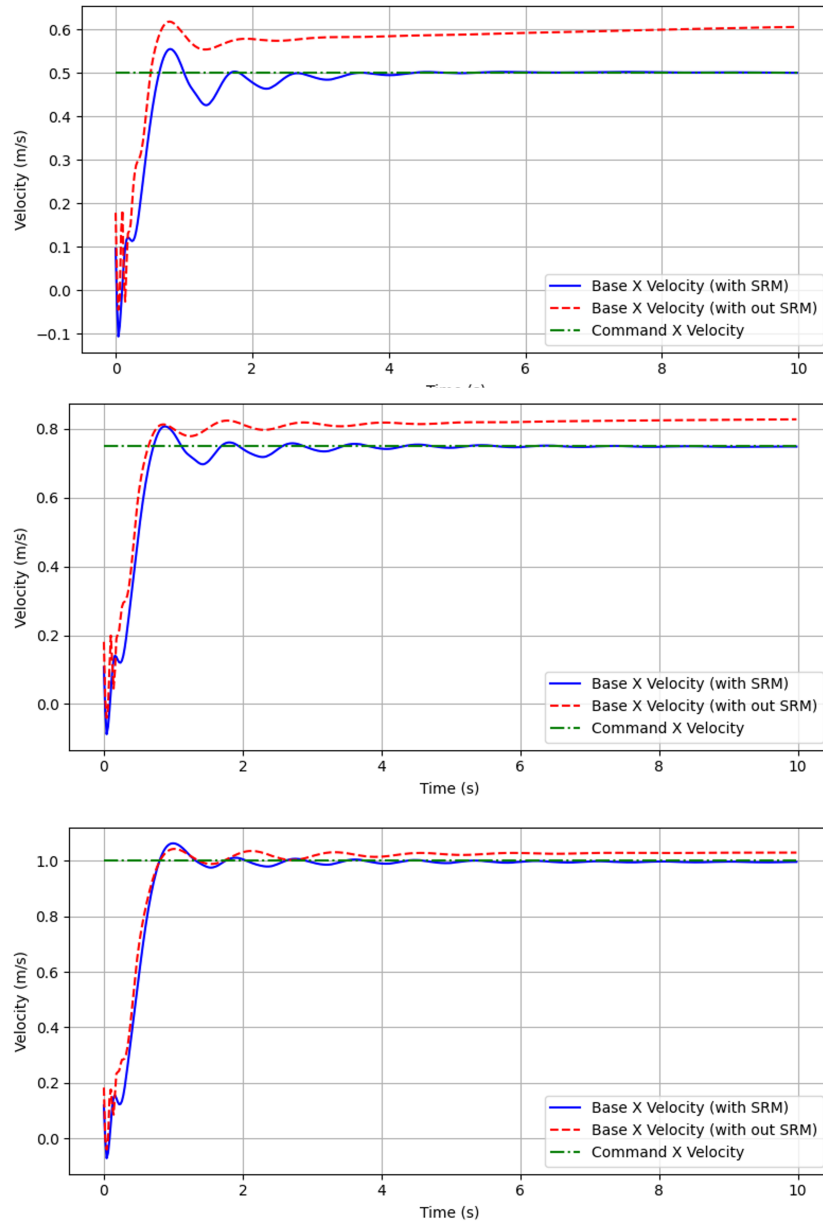


Figure 4.3: Forward velocity response to step input commands with and without SRM (sim-to-sim), showing reduced steady-state error with SRM.

Table 4.2: Real-world forward-velocity tracking errors.

Condition	RMSE	MAE	w_{RMSE}	w_{MAE}
with SRM (proposed)	0.291831	0.230260	0.109146	0.090787
without SRM (baseline)	0.673222	0.649399	0.631776	0.596709

4.2.3 Real-World Velocity Tracking Performance

We evaluate sim-to-real deployment by running the learned policy on the physical robot and estimating forward velocity using the OptiTrack-based procedure described in Section 4.1.2. We report tracking behavior by comparing the commanded forward velocity v_x^{cmd} and the measured response v_x , and summarize performance using standard tracking-error metrics. Full time-series plots for additional trials are provided in Appendix 6.2.

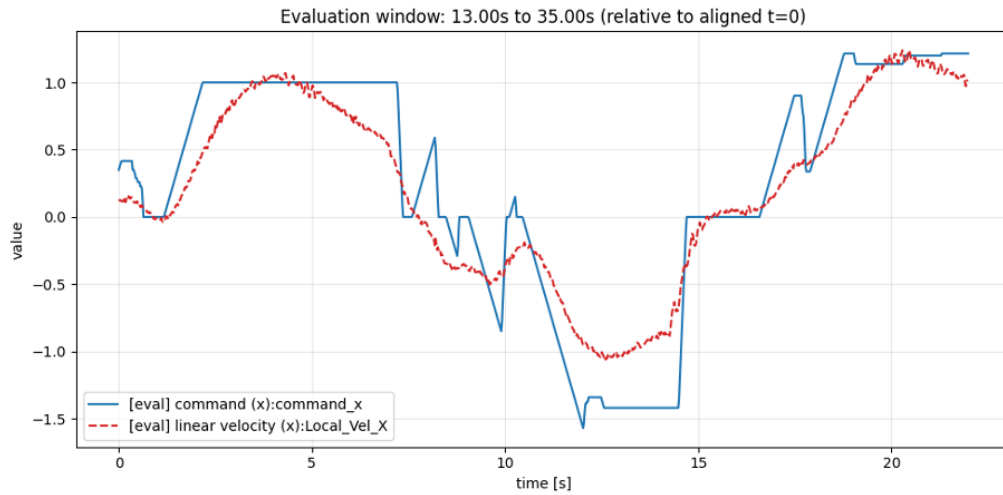
Quantitative comparison

Table 4.2 summarizes real-world tracking errors for the proposed SRM policy and a baseline trained without SRM. Because it is difficult to reproduce perfectly identical command trajectories across hardware runs, our comparison is conducted under the same evaluation protocol (time alignment, evaluation horizon, and metric definitions), rather than requiring trial-wise identical command sequences.

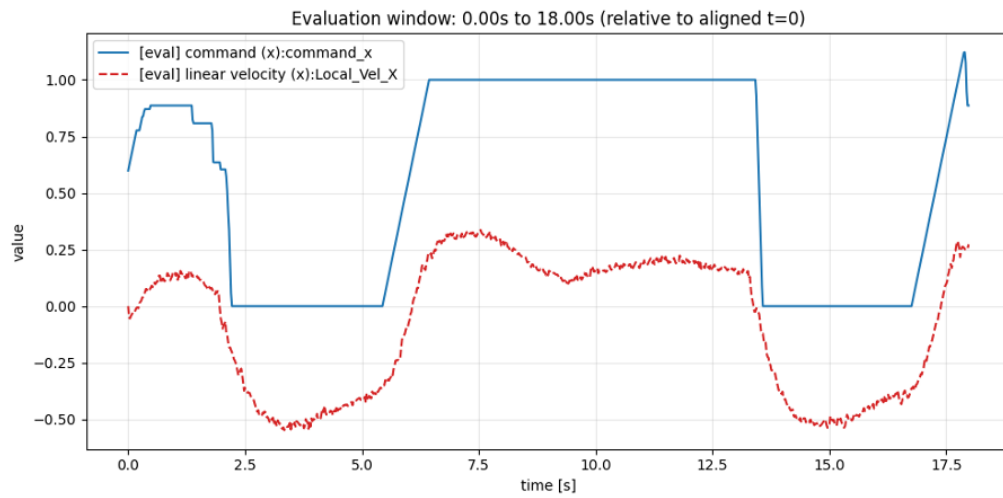
Over the evaluation horizon, SRM reduces RMSE from 0.673222 to **0.291831** and MAE from 0.649399 to **0.230260**. In addition, steady-state tracking performance over the last 2 s improves substantially: w_{RMSE} decreases from 0.631776 to **0.109146** and w_{MAE} decreases from 0.596709 to **0.090787**. These results indicate that SRM improves both overall tracking accuracy and final regulation behavior on hardware.

Qualitative behavior

Figure 4.4 shows representative real-world trials for the SRM-enabled policy and the baseline, including aligned command/response trajectories and the associated error summaries. Although the exact command sequences differ between hardware runs, both trials demonstrate typical step-like command changes and highlight the qualitative difference in tracking behavior. The SRM-enabled policy exhibits smaller tracking deviations and reduced steady-state bias in the presented trial, which is consistent with the quantitative comparison in Table 4.2. Additional real-world plots are provided in Appendix 6.2.



(a) with SRM (proposed).

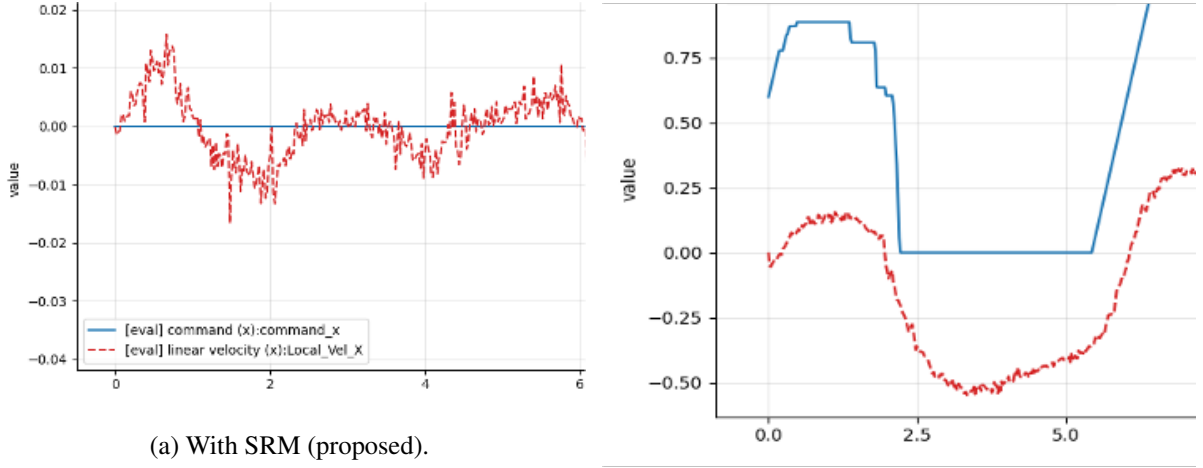


(b) without SRM (baseline).

Figure 4.4: Representative real-world forward-velocity tracking results. Full trajectories for additional trials are provided in Appendix [6.2](#).

Station-keeping at zero command

To further examine stability under zero command, we evaluate station-keeping behavior where the commanded forward velocity is held at $v_x^{\text{cmd}} = 0$. Figure 4.5 shows representative real-world trials. With SRM, the measured forward velocity remains close to zero with small fluctuations, whereas the baseline without SRM exhibits noticeable drift, including sustained backward motion during the zero-command interval.



(a) With SRM (proposed).

(b) Without SRM (baseline).

Figure 4.5: Real-world station-keeping under zero command ($v_x^{\text{cmd}} = 0$).

Table 4.3 summarizes tracking errors. The SRM-enabled policy achieves substantially smaller errors than the baseline, consistent with reduced drift in Fig. 4.5.

Table 4.3: Real-world station-keeping errors under zero command

Condition	RMSE	MAE
with SRM (proposed)	0.013634	0.009517
without SRM (baseline)	0.672472	0.694215

V. Conclusion

In this study, we investigated sim-to-real deep reinforcement learning for hybrid wheeled–legged locomotion under partial observability. We introduced a *State Restoration Model (SRM)* that reconstructs a state-like latent representation from observation histories and used it to condition a Hybrid Locomotion Policy trained in simulation.

Across the proposed *simulation* $\rightarrow$ *simulation* $\rightarrow$ *real* evaluation pipeline, SRM improved commanded-velocity tracking and reduced steady-state bias compared with a baseline trained without SRM. In real-robot experiments, the SRM-enabled policy exhibited smaller tracking errors and more stable station-keeping behavior under near-zero commands, indicating that restoring a state-like representation from temporal context can mitigate ambiguity induced by partial observability and support more reliable transfer from simulation to hardware.

Several limitations suggest concrete directions for future work. The specific contribution of SRM-restored quantities beyond velocity-related effects has not been isolated, particularly because SRM approximates privileged full-state information that can include reward-related signals and contact- or height-related quantities; systematic ablations are needed to identify which restored components are responsible for the observed gains. In addition, the current real-world evaluation reflects nominal operating conditions, so broader stress tests (e.g., payload changes, surface/friction variations, and external perturbations) are required to characterize robustness envelopes and failure modes. Finally, the combination of reconstruction and reward-consistency objectives may introduce sensitivity to reward design and reward-scale mismatch between simulation and reality, motivating studies that reweight or remove the reward-consistency term and perturb reward components to assess behavioral stability.

VI. Appendix

6.1 Training Hyperparameters

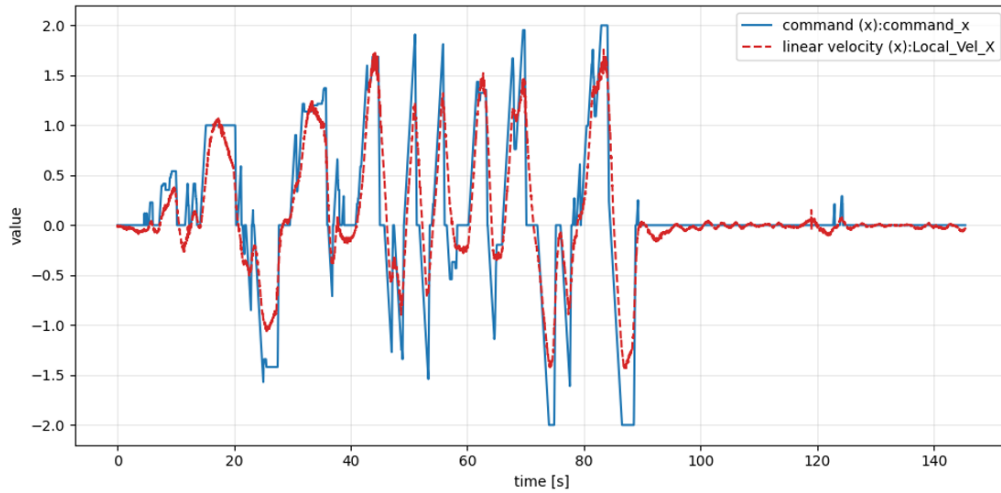
Table 6.1 summarizes the main network architectures and PPO/SRM hyperparameters used in our experiments.

Table 6.1: Key training hyperparameters (network and optimization).

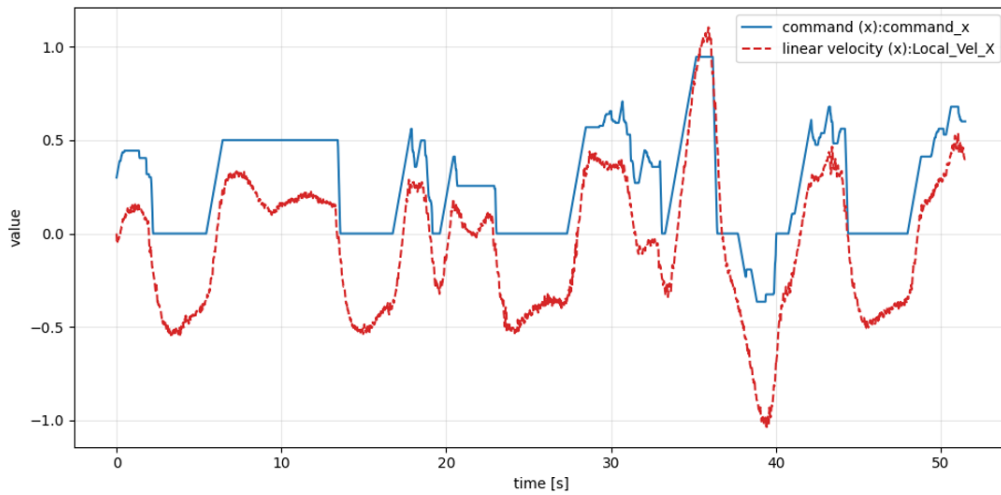
Category	Setting
Random seed	42
Rollout length	<code>num_steps_per_env = 24</code>
Training horizon	<code>max_iterations = 5000</code>
Policy/Value backbone	Actor-Critic MLP (ActorCritic)
Actor hidden dims	(512, 256, 128)
Critic hidden dims	(512, 256, 128)
Activation	ELU
Policy init noise	<code>init_noise_std = 1.0</code>
Algorithm	SRM-PPO (SRMPP0)
Learning rate	1×10^{-3} (adaptive schedule)
Discount / GAE	$\gamma = 0.99$, $\lambda = 0.95$
PPO clipping	<code>clip_param = 0.2</code>
Entropy coef.	0.01
Value loss coef.	1.0
Optimization	5 epochs, 4 mini-batches
KL target	<code>desired_kl = 0.01</code>
Grad clipping	<code>max_grad_norm = 1.0</code>
SRM backbone	GRU (<code>srm_net=gru</code>)
SRM input dims	<code>srm_input_dim=32</code>
SRM hidden dim	256
SRM output dim	5
SRM layers	1
History stacking	<code>num_policy_stacks=2</code> , <code>num_critic_stacks=2</code>

6.2 Real-World Full Trajectories

This appendix provides additional full time-series trajectories for real-world experiments, supplementing the representative results shown in Section 4.2



(a) With SRM (proposed).



(b) Without SRM (baseline).

Figure 6.1: Full real-world trajectories used for qualitative reference.

6.3 Pseudocode for SRM-PPO Training

Algorithm 1 summarizes the training loop of the proposed SRM-PPO framework. At each iteration, we collect rollouts in simulation using the actor policy $\pi_\theta(a_t \mid s_t^o)$, where s_t^o denotes the partially observable input available at deployment. During training, the critic additionally receives privileged full-state information s_t^f that is accessible only in simulation. After collecting T steps, we compute returns R_t and advantages $\hat{A}_t$ using generalized advantage estimation (GAE) with (γ, λ) .

During optimization, each mini-batch performs two coupled updates. First, the SRM parameters ψ are updated to reconstruct a privileged, full-state approximation $\hat{s}_t^f$ from an observation history $s_{t-k:t}^o$. The SRM objective combines a reconstruction loss that aligns $\hat{s}_t^f$ with s_t^f and a reward-consistency loss that aligns the reward computed from the reconstructed state with the reward computed from the privileged state:

$$\mathcal{L}_{\text{SRM}} = \mathcal{L}_{\text{reconstruction}}(\hat{s}_t^f, s_t^f) + \mathcal{L}_{\text{reward}}(R(\hat{s}_t^f, a_t), R(s_t^f, a_t)).$$

Second, the policy/value parameters θ are updated via PPO using the clipped surrogate objective, while the value function is trained to regress the return R_t using the critic input s_t^f during training. This alternating SRM update and PPO update encourages the actor to operate on a restored state representation while retaining the stability benefits of PPO.

Algorithm 1 SRM-PPO Training

Require: policy/value parameters θ , SRM parameters ψ , rollout horizon T , PPO hyperparameters $(\gamma, \lambda, \epsilon)$

```
1: while training do
2:   Initialize rollout buffer  $\mathcal{D} \leftarrow \emptyset$ 
3:   for  $t = 1$  to  $T$  do
4:     Observe actor input  $s_t^o$  (partial) and critic input  $s_t^f$  (privileged, sim only)
5:     Sample  $a_t \sim \pi_\theta(\cdot | s_t^o)$  and execute in simulator
6:     Receive reward  $r_t$  and done flag  $d_t$ 
7:     Store  $(s_t^o, s_t^f, a_t, r_t, d_t, \log \pi_\theta(a_t | s_t^o))$  in  $\mathcal{D}$ 
8:   end for
9:   Compute returns  $R_t$  and advantages  $\hat{A}_t$  from  $\mathcal{D}$  using GAE( $\gamma, \lambda$ )
10:  for each mini-batch  $\mathcal{B} \subset \mathcal{D}$  for several epochs do
    SRM update
11:     $\hat{s}_t^f \leftarrow \text{SRM}_\psi(s_{t-k:t}^o)$ 
12:     $\mathcal{L}_{\text{reconstruction}} \leftarrow \mathcal{L}_{\text{Huber}}(\hat{s}_t^f, s_t^f)$ 
13:     $\mathcal{L}_{\text{reward}} \leftarrow \left\| R(s_t^f, a_t) - R(\hat{s}_t^f, a_t) \right\|^2$ 
14:     $\mathcal{L}_{\text{SRM}} \leftarrow \mathcal{L}_{\text{reconstruction}} + \mathcal{L}_{\text{reward}}$ 
15:    Update  $\psi$  by minimizing  $\mathcal{L}_{\text{SRM}}$ 
    PPO update
16:     $\rho \leftarrow \exp(\log \pi_\theta(a | s^o) - \log \pi_{\text{old}}(a | s^o))$ 
17:     $\mathcal{L}_\pi \leftarrow \mathbb{E} \left[ \max \left( -\rho \hat{A}, -\text{clip}(\rho, 1 - \epsilon, 1 + \epsilon) \hat{A} \right) \right]$ 
18:     $\mathcal{L}_v \leftarrow \mathbb{E} \left[ (V_\theta(s^f) - R)^2 \right]$ 
19:    Update  $\theta$  by minimizing  $\mathcal{L}_\pi + \mathcal{L}_v$ 
20:  end for
21: end while
```

요약문

실세계 로봇에서 보행 정책을 안정적으로 실행하는 것은 제한된 센서 관측으로 인해 상태를 완전하게 관측하기 어렵고, 시뮬레이션에서 학습한 정책이 시뮬레이션-현실 격차로 인해 실제 하드웨어에서 성능이 저하될 수 있다는 점에서 여전히 도전적이다. 본 논문은 바퀴와 다리를 결합한 하이브리드 주행을 수행하는 이륜-이족 로봇을 대상으로, 시뮬레이션에서 학습한 심층 강화학습 정책을 실제 로봇에 전이하기 위한 방법을 다룬다. 이 문제는 바퀴-다리 결합 동역학과 간헐적 접촉으로 인해 부분가시성 특성이 두드러지며, 단일 시점의 관측만으로는 안정적인 제어에 필요한 마르코프 상태를 구성하기 어렵다.

이를 해결하기 위해, 관측 이력으로부터 상태에 준하는 잠재표현을 복원하는 상태 복원 모델(State Restoration Model, SRM)을 제안한다. SRM은 시뮬레이션에서만 이용 가능한 특권 정보(전체 상태)를 근사하도록 학습되며, 비대칭 액터-크리틱(asymmetric actor-critic) 학습 구조에 통합된다. 학습 시 크리틱은 특권 정보를 활용해 정책을 평가하고, 액터는 SRM이 복원한 표현을 입력으로 받아 실제 환경에서도 일관된 의사결정을 수행하도록 한다.

제안 방법은 FlaminGO 8자유도 이륜-이족 로봇에서 검증하였다. Isaac Sim 에서 정책을 학습한 뒤, 독립 물리 엔진인 MuJoCo에서 교차 시뮬레이터 검증을 수행하고, 최종적으로 실제 로봇에 배포하는 시뮬레이션 → 시뮬레이션 → 현실 절차로 평가하였다. 실험 결과 SRM을 적용한 정책은 SRM이 없는 기준 방법 대비 속도 명령 추종 성능이 향상되고 정상상태 편향이 감소하였으며, 특히 영(0)에 가까운 명령에서의 정지 유지 동작이 더 안정적으로 나타났다. 이는 시간적 문맥을 활용해 상태에 준하는 표현을 복원하는 접근이 부분가시성으로 인한 불확실성을 완화하고, 하이브리드 주행에서의 시뮬레이션-현실 전이 강건성을 향상시킬 수 있음을 시사한다.

References

- [1] Kaelbling, Leslie Pack, Michael L. Littman, and Anthony R. Cassandra. “Planning and acting in partially observable stochastic domains.” *Artificial Intelligence* 101(1–2): 99–134 (1998).
- [2] Tobin, Josh, et al. “Domain randomization for transferring deep neural networks from simulation to the real world.” In *Proceedings of the 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (2017).
- [3] Peng, Xue Bin, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. “Sim-to-Real Transfer of Robotic Control with Dynamics Randomization.” In *Proceedings of the 2018 IEEE International Conference on Robotics and Automation (ICRA)* (2018).
- [4] Tan, Jie, et al. “Sim-to-Real: Learning Agile Locomotion For Quadruped Robots.” In *Robotics: Science and Systems (RSS)* (2018).
- [5] Hwangbo, Jemin, et al. “Learning agile and dynamic motor skills for legged robots.” *Science Robotics* 4(26): eaau5872 (2019).
- [6] Bjelonic, Marko, Victor Klemm, Joonho Lee, et al. “A Survey of Wheeled-Legged Robots.” In *Robotics in Natural Settings, Lecture Notes in Networks and Systems*, vol. 530, pp. 83–94. Springer (2023).
- [7] Grandia, Ruben, et al. “Feedback MPC for torque-controlled legged robots.” In *Proceedings of the 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (2019).
- [8] Ding, Yanran, et al. “Representation-free model predictive control for dynamic motions in quadrupeds.” *IEEE Transactions on Robotics* 37(4): 1154–1171 (2021).
- [9] Schulman, John, et al. “Proximal policy optimization algorithms.” arXiv preprint arXiv:1707.06347 (2017).
- [10] Hausknecht, Matthew, and Peter Stone. “Deep Recurrent Q-Learning for Partially Observable MDPs.” arXiv preprint arXiv:1507.06527 (2015).
- [11] Biswal, Priyaranjan, and Prases K. Mohanty. “Development of quadruped walking robots: A review.” *Ain Shams Engineering Journal* 12(2): 2017–2031 (2021).

- [12] Hutter, Marco, et al. “Anymal-a highly mobile and dynamic quadrupedal robot.” In *Proceedings of the 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (2016).
- [13] Shin, Young-Ha, et al. “Design of KAIST HOUND, a quadruped robot platform for fast and efficient locomotion with mixed-integer nonlinear optimization of a gear train.” In *Proceedings of the 2022 IEEE International Conference on Robotics and Automation (ICRA)* (2022).
- [14] RobStride. *Product Information*. GitHub repository, accessed December 24, 2025. https://github.com/RobStride/Product_Information.
- [15] Miki, Takahiro, et al. “Learning robust perceptive locomotion for quadrupedal robots in the wild.” *Science Robotics* 7(62): eabk2822 (2022).
- [16] Pinto, Lerrel, et al. “Asymmetric actor critic for image-based robot learning.” arXiv preprint arXiv:1710.06542 (2017).
- [17] Mittal, Mayank, Pascal Roth, et al. “Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning.” arXiv preprint arXiv:2511.04831 (2025).
- [18] Todorov, Emanuel, Tom Erez, and Yuval Tassa. “MuJoCo: A physics engine for model-based control.” In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 5026–5033 (2012).
- [19] Burden, Richard L., and J. Douglas Faires. *Numerical Analysis*. 9th ed., Brooks/Cole, Cengage Learning (2010).

Acknowledgements

석사 학위 논문의 마침표를 찍으며, 지난 2년여의 학위 과정을 되돌아봅니다. 연구자로서의 첫걸음을 떼던 순간의 설렘과 막막함이 엇그제 같은데, 어느덧 이 과정을 마무리하는 시점에 서게 되었습니다. 학문의 깊이를 더하고자 부단히 노력했으나 여전히 부족함을 느끼기에 아쉬움이 남습니다. 하지만 그 부족함을 채워주시고 저를 성장시켜 주신 많은 분의 도움이 있었기에 이 결실을 맺을 수 있었습니다. 이 지면을 빌려 감사의 마음을 전하고자 합니다.

먼저, 연구에 대한 깊은 통찰력과 따뜻한 인품으로 부족한 제게 학문의 길을 열어주신 한수희 교수님께 깊은 존경과 감사의 말씀을 올립니다. 교수님께서 보여주신 연구에 대한 열정과 세심한 지도 편달은 제가 연구자로서 갖춰야 할 태도를 배우는 큰 밑거름이 되었습니다. 앞으로 나아갈 길에서도 교수님의 가르침을 잊지 않고 정진하는 연구자가 되겠습니다. 또한, 바쁘신 일정 중에도 흔쾌히 학위 심사를 맡아주시고, 제 연구가 한 단계 발전할 수 있도록 아낌없는 조언을 해주신 김상우 교수님과 박부건 교수님께도 진심으로 감사드립니다.

동고동락하며 연구실 생활의 큰 힘이 되어준 CoCEL 식구들에게도 고마운 마음을 전합니다. 연구 내외적으로 든든한 버팀목이자 코셀로의 CTO 인공지능 대가 장원 형, 코셀로 이사이자 저의 소중한 포항 메이트 상현이, AI팀 팀장이자 든든한 버팀목인 종수 형, 너무나 착한 지석이 형, 함께 밤을 지새우며 도와주었던 민재와 준영이, 수업의 든든한 버팀목이 되어준 제어 전문가 동형이, 로봇으로 마음이 통하는 수학자 민종이, 마지막 실험까지 발 벗고 도와준 매니폴드 장인 진우, 함께한 추억이 많은 승현 형, 드론팀 팀장이자 든든한 인간 GPT 지호, 그리고 연구실에 새로운 활력을 불어넣어 주는 성민, 준호, 근영, 혜령, 든든한 신일 형님 등 모든 동료 덕분에 힘들었던 순간들도 웃으며 이겨낼 수 있었습니다. 서로 다른 연구 분야임에도 세미나를 통해 귀한 아이디어를 나누고 격려해 준 드론팀, SLAM팀, 에너지팀 분들에게도 감사를 표합니다.

마지막으로, 저를 향한 무한한 믿음과 사랑으로 언제나 제 편이 되어주시는 아버지, 어머니께 말로는 다 할 수 없는 감사의 마음을 바칩니다. 그리고 항상 묵묵히 저를 응원해 주는 우리 형에게도 고마움을 전합니다. 우리 가족의 앞날에 건강과 행복이 가득하기를 기원합니다. 가족들의 헌신적인 지지가 있었기에 흔들리지 않고 학업에 매진할 수 있었습니다. 언제나 든든한 아들이자 동생이 되어 그 사랑에 보답하겠습니다.

제 곁을 지켜준 모든 소중한 인연의 행복을 바라며, 이 논문을 바칩니다.



Curriculum Vitae

Name : Jae Hyung Cho

Education

2024. 03. – 2026. 02. Department of Convergence IT Engineering, Pohang University of Science and Technology (M.S.)
2022. 03. – 2023. 02. Department of Mechatronics Engineering, Dealim University (B.S.)
2017. 03. – 2022. 02. Department of Automotive, Dealim University (P.B.S.)

Experience

2025. 04. – CEO, COCELO Inc.
2025. 01. – 2025. 12. Smart Manufacturing Innovation R&D Project (AI-based Steel Structure Production Management), Ministry of SMEs and Startups.
2024. 09. – 2025. 09. Strategic Industry-Academia Project, POSTECH-Samsung Electronics Industry-Academia Research Center.
2024. 05. – 2025. 06. Development of AI-based Grab Type Ship Unloader (GTSU), Research Institute of Industrial Science and Technology (RIST).

Affiliation

Computational Control Engineering Lab. (CoCEL), Department of Convergence IT Engineering, POSTECH, South Korea.



